StockSense AI

Deep Technical LAN Architecture & Security Engine

This document provides a comprehensive, bottom-up engineering explanation of how StockSense AI transforms a standalone Windows laptop into a private, multi-device financial terminal server, how network packets travel across the Wi-Fi, how Windows Firewall filtering is handled, and how the Zero-Trust device authentication pipeline operates at the socket, middleware, and database layers.

1. Network Layer: Sockets, IP Binding & Subnet Routing

The Socket Binding Problem (`127.0.0.1` vs `0.0.0.0`)

By default, development servers (like `uvicorn main:app` or `next dev`) bind their listening sockets to the loopback interface (`127.0.0.1` or `localhost`).

- `127.0.0.1` **(Loopback Interface)**: The operating system routes these network packets internally through a virtual software loopback adapter. The physical Wi-Fi or Ethernet network interface controller (NIC) is completely bypassed. Any TCP SYN packet arriving from an external device on the Wi-Fi is ignored at the kernel level because no socket is listening on that interface.
- `0.0.0.0` **(**`INADDR_ANY`**)**: To allow other devices on the LAN to connect, both FastAPI and Next.js are explicitly instructed to bind to `0.0.0.0`:
- FastAPI: `uvicorn app.main:app --host 0.0.0.0 --port 8000`
- Next.js: `next dev -H 0.0.0.0 -p 3000` (or `next start -H 0.0.0.0 -p 3000`)
- When a process binds to `0.0.0.0`, the Windows kernel socket manager registers the listening TCP port on **all** local network interfaces simultaneously: Loopback (`127.0.0.1`), Wi-Fi adapter (e.g., `192.168.1.243`), Ethernet, and Virtual Switch adapters.

Detailed Network Segment Determination (Subnet Mask Bitwise AND & Routing Table)

Before transmitting a single radio wave or network packet, the client device (e.g. smartphone at `192.168.1.55`) must answer a fundamental Layer 3 routing question: **Is the destination (`192.168.1.243`) located directly on the local physical network (on-link), or does it require routing through the default gateway (off-link)?**

1. The Kernel Routing Table & Longest Prefix Match (LPM)

When an application calls `connect()` or `sendto()`, the OS network stack (e.g. `tcpip.sys` on Windows, Darwin kernel on iOS, or Linux kernel on Android) consults its internal Routing Information Base (RIB). A typical host routing table contains:

Destination Network	Netmask / CIDR	Gateway	Interface	Metric	Routing Scope
127.0.0.1	255.255.255.255 (/32)	On-link	Loopback (127.0.0.1)	1	Local Host Process
192.168.1.55	255.255.255.255 (/32)	On-link	Wi-Fi (192.168.1.55)	25	Local Adapter IP
192.168.1.0	**255.255.255.0 (/24)**	**On-link**	**Wi-Fi (192.168.1.55)**	**25**	**Direct Local Subnet**
192.168.1.255	255.255.255.255 (/32)	On-link	Wi-Fi (192.168.1.55)	25	Subnet Broadcast
224.0.0.0	240.0.0.0 (/4)	On-link	Wi-Fi (192.168.1.55)	25	Multicast Group

Destination Network	Netmask / CIDR	Gateway	Interface	Metric	Routing Scope
0.0.0.0	0.0.0.0 (/0)	192.168.1.1	Wi-Fi (192.168.1.55)	35	Default Gateway (Internet)

The kernel evaluates matching routes using **Longest Prefix Match (LPM)**: the route with the most specific subnet mask (highest prefix length) that matches the target IP is selected.

2. The Bitwise Logical AND Calculation

To test if a target matches the Direct Local Subnet route (`192.168.1.0/24`), the kernel executes a hardware-level **bitwise logical AND operation** between the 32-bit IPv4 address of the host, the 32-bit IPv4 address of the target, and the 32-bit Subnet Mask:

```
FORMULA:
  Network_ID = IP_Address BITWISE_AND Subnet_Mask
  IS_LOCAL   = (Host_IP & Subnet_Mask) == (Target_IP & Subnet_Mask)
```

Bit-by-Bit Binary Execution:

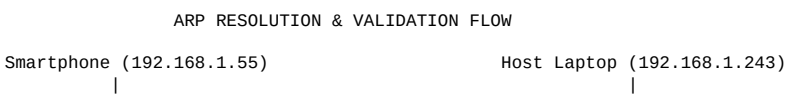
```
Step 1: Compute Client Local Network ID
  Client Host IP:  192.168.1.55     ->  11000000 . 10101000 . 00000001 . 00110111
  Subnet Mask:     255.255.255.0    ->  11111111 . 11111111 . 11111111 . 00000000
  --------------------------------------------------------------------------------
  BITWISE AND:     192.168.1.0      ->  11000000 . 10101000 . 00000001 . 00000000

Step 2: Compute Target Server Network ID
  Target Host IP:  192.168.1.243    ->  11000000 . 10101000 . 00000001 . 11110011
  Subnet Mask:     255.255.255.0    ->  11111111 . 11111111 . 11111111 . 00000000
  --------------------------------------------------------------------------------
  BITWISE AND:     192.168.1.0      ->  11000000 . 10101000 . 00000001 . 00000000

Step 3: Equality Comparator
  Client Network ID (192.168.1.0) == Target Network ID (192.168.1.0)  ===>  TRUE
```

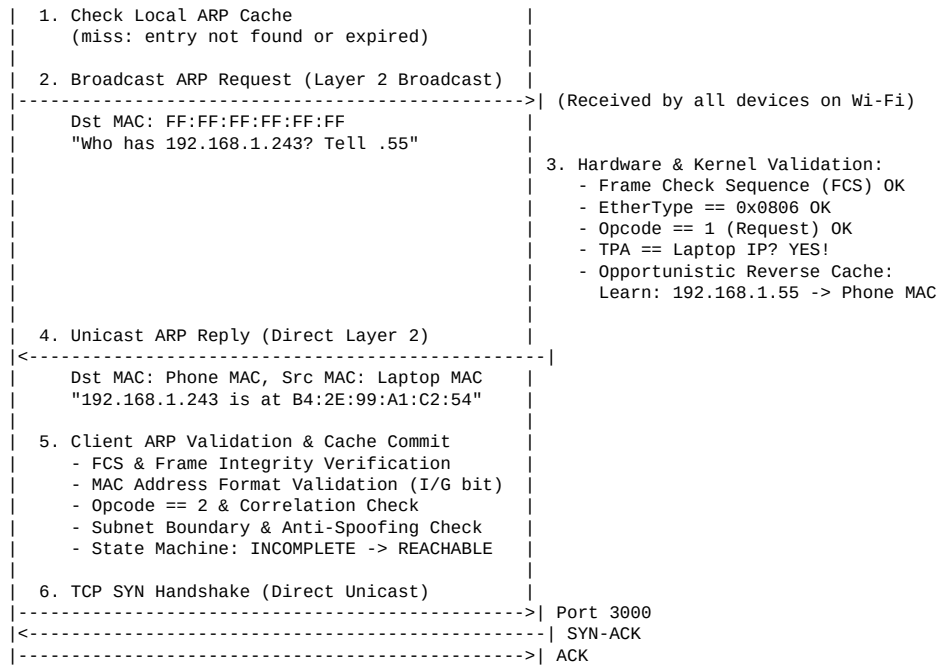
3. Routing Decision Branches: On-Link vs Off-Link

- **Branch A: Destination is ON-LINK (Direct Delivery - StockSense AI LAN case)**:
- Because `Client Net ID == Target Net ID`, the target device is physically reachable on the same Layer 2 broadcast domain (Wi-Fi SSID or switch).
- **Action**: The kernel routes the packet directly to the local network interface. The Next-Hop IP is set to the destination IP (`192.168.1.243`).
- **Layer 2 Requirement**: The client must resolve the **laptop's MAC address** using an ARP broadcast.
- **Branch B: Destination is OFF-LINK (Indirect Delivery - e.g. querying Google at `142.250.190.46`)**:
- `(142.250.190.46 & 255.255.255.0) = 142.250.190.0 != 192.168.1.0`.
- The direct route does not match. The fallback default route `0.0.0.0/0` is selected.
- **Action**: The Next-Hop IP is set to the **Default Gateway** (`192.168.1.1`).
- **Layer 2 Difference**: The client does **NOT** resolve the server's MAC. It resolves the **Router/Gateway's MAC address**, setting the outer Ethernet destination to the router while leaving the inner Layer 3 IP header set to `142.250.190.46`.

ARP Protocol Architecture, MAC Acquisition & Validation Deep-Dive

Because IPv4 operates at Layer 3 (logical addressing) while Wi-Fi (IEEE 802.11) and Ethernet (IEEE 802.3) operate strictly at Layer 2 (physical addressing), Layer 3 IP datagrams cannot be transmitted across physical media without an encapsulation translation. The **Address Resolution Protocol (ARP - RFC 826)** bridges this gap.





A. How the ARP Table Takes and Learns MAC Addresses

Operating systems populate their ARP cache (managed by `tcpip.sys` on Windows and the Neighbor Discovery subsystem in the Linux/Android/Darwin kernels) through two distinct mechanisms:

1. Active Resolution (Solicited Learning)

When a device needs to send an IP packet to an unknown local IP, it initiates active resolution:

- The kernel creates an entry in the ARP table in an `INCOMPLETE` state.
- The kernel transmits an ARP Request packet.
- Upon receiving a matching unicast ARP Reply, the kernel updates the entry with the sender's MAC address and transitions the state to `REACHABLE` or `VALID`.

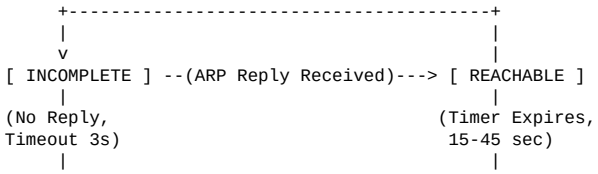
2. Passive Snooping & Piggybacking (Opportunistic Learning)

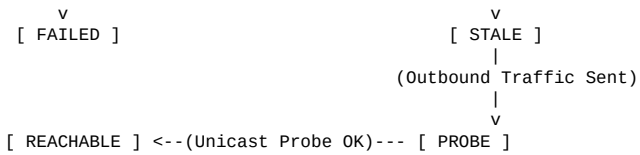
RFC 826 dictates that whenever an ARP packet (Request or Reply) is received by a host, if the host already has an entry for the sender's IP address, or if the packet is targeted directly at the host:

- The receiving host extracts the **Sender Protocol Address (SPA)** (`192.168.1.55`) and the **Sender Hardware Address (SHA)** (`AA:BB:CC:DD:EE:FF`).
- The Windows laptop **automatically updates or inserts** this mapping into its ARP cache (`netsh interface ip show neighbors`).
- **Performance Benefit**: When the laptop later needs to send data back to the phone (e.g. sending the HTTP response on port 3000), it **does not need to broadcast a reverse ARP request**. It already knows the phone's MAC from the initial ARP request!

3. The Kernel Neighbor Cache State Machine (RFC 4861 / Windows NUD)

In modern operating systems, ARP cache entries are not static; they follow a strict state machine:

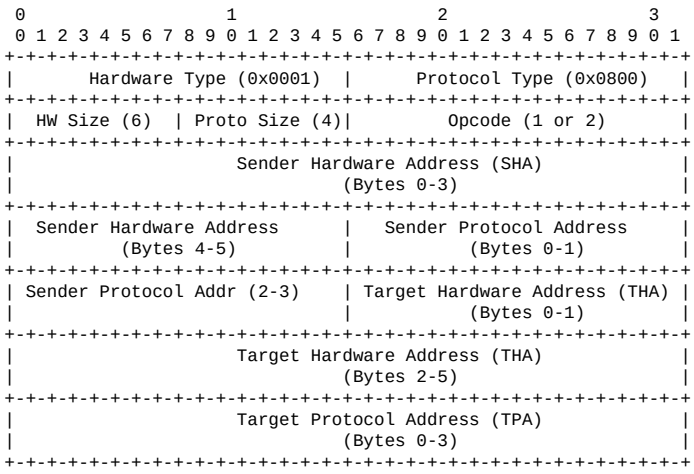




- `INCOMPLETE`: An ARP request has been broadcast; the kernel is waiting for a reply. Outbound packets for this IP are temporarily buffered in a ring queue.
- `REACHABLE`: The MAC address has been validated and confirmed reachable within the reachability timer (typically 15 to 45 seconds on Windows, up to 300 seconds on mobile OS). Traffic flows with zero ARP latency.
- `STALE`: The reachability timer expired without recent traffic. The entry is kept and can still be used for immediate transmission, but sending traffic triggers a state change to `DELAY`/`PROBE`.
- `PROBE`: The kernel sends unicast ARP requests (probes) directly to the cached MAC to verify that the device hasn't disconnected, powered off, or changed Wi-Fi IP. If acknowledged, it returns to `REACHABLE`.
- `FAILED`: If probes fail after maximum retries (typically 3 attempts), the entry is purged and subsequent connection attempts return `EHOSTUNREACH` or `Destination Host Unreachable`.

B. The Complete Anatomy of ARP Packets

The ARP packet sits directly inside the Layer 2 Ethernet / 802.11 frame payload:



- **Hardware Type (`0x0001`)**: Defines the physical link layer (Ethernet / 802.11 Wi-Fi).
- **Protocol Type (`0x0800`)**: Defines the networking layer protocol being resolved (IPv4).
- **Hardware Size (`0x06`)**: MAC addresses are 6 octets (48 bits).
- **Protocol Size (`0x04`)**: IPv4 addresses are 4 octets (32 bits).
- **Opcode**:
- `0x0001` = `ARPREQUEST` ("Who has this IP? Send MAC to me").
- `0x0002` = `ARPREPLY` ("I have this IP; here is my MAC").

C. How Devices Validate ARP Packets and MAC Addresses

ARP was originally designed in 1982 without cryptographic authentication. To protect against corrupted frames, invalid network states, and malicious ARP spoofing/poisoning, modern network stacks (Windows TCP/IP driver, Linux `arp_filter`, Android, iOS) execute **5 layers of rigorous validation** before committing an entry to the ARP table:

1. Physical Layer & Frame Check Sequence (FCS) Validation

- **Hardware CRC32 Check**: The Wi-Fi / Ethernet Network Interface Card (NIC) verifies the 32-bit Cyclic Redundancy Check (CRC) at the end of the frame. If radio interference or packet corruption altered even a single bit during Wi-Fi transmission, the physical NIC drops the frame instantly without waking the OS CPU.

2. MAC Address Structure & Bit-Level Validation

When an ARP frame is parsed, the kernel validates the format of the **Sender Hardware Address (SHA)**:

- **Individual/Group (I/G) Bit Check**:
- The least significant bit (LSB) of the first octet of the MAC address indicates whether the address is **Unicast (0)** or **Multicast/Broadcast (1)**:

```
Octet 0:  b7  b6  b5  b4  b3  b2  b1  b0 (I/G bit)
                                       |--- 0 = Individual (Unicast)
                                            1 = Group (Multicast/Broadcast)
```

- **Validation Rule**: A sender hardware address **must have** `I/G == 0`. If an ARP reply claims a sender MAC of `FF:FF:FF:FF:FF:FF` or `01:00:5E:...` (multicast), the kernel **rejects and drops the packet**, preventing broadcast storm loops and cache corruption.
- **Null MAC Check**: Addresses with all zeros (`00:00:00:00:00:00`) are rejected as invalid hardware descriptors.
- **Universal vs Locally Administered (U/L) Bit Check**:
- The second least significant bit of octet 0 defines whether the MAC is globally unique (burned in at factory) or locally administered (randomized).
- Modern smartphones (iOS "Private Wi-Fi Address", Android "Use randomized MAC") randomize this address per Wi-Fi SSID for privacy. The OS kernel permits locally administered MACs (`U/L == 1`) as long as the I/G bit is 0 (unicast).

3. Protocol Header Sanity & Opcode Validation

The receiving network stack verifies:

- `Hardware Type == 1` (Ethernet/Wi-Fi).
- `Protocol Type == 0x0800` (IPv4).
- `Hardware Size == 6` AND `Protocol Size == 4`.
- `Opcode == 1` (Request) OR `Opcode == 2` (Reply). Any unknown or unsupported opcodes (e.g. RARP, DRARP) are discarded.

4. State Correlation & Solicitation Verification

- When the smartphone receives an ARP Reply (`Opcode == 2`):
- It checks whether an outstanding ARP resolution request was recently transmitted for `SPA (192.168.1.243)` within its solicitation window (typically 1-3 seconds).
- Unsolicited Gratuitous ARP (GARP) replies: If a device announces an IP without a prior request, modern OS kernels inspect if the IP already exists in the cache:
- If the IP does not exist, unsolicited entries are often **ignored** (preventing malicious attackers from filling the cache with bogus entries).
- If the IP already exists, the kernel may update the MAC or issue an active unicast verification probe before updating, guarding against ARP poisoning (Man-In-The-Middle attacks).

5. Subnet Boundary Cross-Check

- The kernel compares the `Sender Protocol Address (SPA)` against the network mask of the receiving interface:

```
Is (SPA & Interface_Netmask) == (Interface_IP & Interface_Netmask)?
```

- If an ARP packet arrives on the Wi-Fi interface (`192.168.1.55/24`) claiming to represent an IP from a foreign subnet (e.g. `10.0.0.88` or `172.16.4.1`), the packet is **flagged as illegal and dropped**. ARP cannot resolve cross-subnet addresses; those must traverse a default gateway.

6. Wi-Fi Access Point Layer Validation (AP Isolation & Client Isolation)

- On standard home and private office Wi-Fi routers, the 802.11 Access Point forwards Layer 2 broadcast frames (`FF:FF:FF:FF:FF:FF`) to all associated wireless stations, enabling StockSense AI discovery.
- **Gotcha / Edge Case**: On enterprise guest Wi-Fi networks or public cafes with **"AP Isolation"** or **"Client Isolation"** enabled, the Wi-Fi router's firmware intentionally suppresses peer-to-peer ARP and unicast frames between wireless clients. On such networks, devices cannot see each other even if they are in the same IP subnet. Connecting both devices to the same private home/office Wi-Fi or creating a mobile hotspot bypasses AP isolation completely.

2. Windows Defender Firewall & Kernel Bypass Architecture

The Windows Filtering Platform (WFP)

Windows includes a deep kernel-level packet inspection framework called the **Windows Filtering Platform (WFP)**.

- For traffic originating on `127.0.0.1`, WFP recognizes the loopback IPC traffic and allows it unconditionally.
- For incoming Wi-Fi traffic (`192.168.1.X`), the packet arrives at the NDIS (Network Driver Interface Specification) miniport driver. Windows Defender Firewall checks its active rules. If no rule explicitly permits inbound TCP traffic on port 3000 or 8000, **Windows silently drops the packet with `TCP RST` or drops it silently (connection timeout)**.

How We Configure the Firewall (Bypass via Whitelisting)

Rather than unsafely disabling the firewall entirely, we use the Windows NetSecurity API via PowerShell (`setup_windows_firewall.ps1`) or `netsh` (`setup_firewall.bat`):

```
$ruleParams = @{
    DisplayName = "StockSense-AI-LAN-Server"
    Direction   = "Inbound"
    Action      = "Allow"
    Protocol    = "TCP"
    LocalPort   = @("3000", "8000")
    Profile     = "Private"
    Description = "StockSense AI Native LAN Server - Unified Frontend (3000) and Backend (8000)"
}
New-NetFirewallRule @ruleParams
```

Why the `Profile = "Private"` Setting is Critical for Security

Windows categorizes network adapters into three firewall profiles:

- **Domain**: Corporate Active Directory networks.
- **Private**: Home or trusted office Wi-Fi networks.
- **Public**: Coffee shops, airports, hotel Wi-Fi.

By restricting our inbound firewall rule strictly to `Profile = "Private"`, **your laptop remains 100% shielded when connected to public Wi-Fi networks**. The firewall rule automatically deactivates on public networks and only opens when your laptop is connected to your trusted private home/office Wi-Fi.

3. Reverse Proxy & Single-Port Gateway Architecture

In standard web architectures, running a frontend on port 3000 and a backend on port 8000 creates two major problems:

- **CORS (Cross-Origin Resource Sharing)**: When a phone loads HTML/JS from `http://192.168.1.243:3000` and attempts an `XMLHttpRequest` or `fetch()` to `http://192.168.1.243:8000`, mobile browsers block the request unless strict, complex pre-flight CORS headers are negotiated.
- **Double Port Exposure**: You would have to open and manage two separate external ports on the network.

The Solution: Next.js Unified Ingress Gateway

In `frontend/next.config.js`, we implemented native reverse-proxy rewrites:

```
module.exports = {
  async rewrites() {
    return [
      {
        source: "/api/v1/:path*",
        destination: "http://127.0.0.1:8000/api/v1/:path*",
      },
      {
        source: "/docs",
        destination: "http://127.0.0.1:8000/docs",
      },
      {
        source: "/openapi.json",
        destination: "http://127.0.0.1:8000/openapi.json",
      },
    ];
  },
};
```

How Network Routing Works with This Gateway:

- The remote phone opens `http://192.168.1.243:3000`. Next.js serves the HTML, CSS, and React bundles over port 3000.
- When the phone's browser calls an API endpoint (e.g. `/api/v1/market/quote/AAPL`), it sends the request to `http://192.168.1.243:3000/api/v1/market/quote/AAPL`.
- Next.js receives the request on port 3000 and proxies it over internal localhost loopback to `http://127.0.0.1:8000/api/v1/market/quote/AAPL`.
- FastAPI handles the request and sends the response back to Next.js, which streams it to the phone.
- **Zero CORS issues**: The browser communicates only with origin `http://192.168.1.243:3000`.
- **Single Port Simplicity**: Port 8000 never needs to be exposed to external devices.

4. Client-Side Device Fingerprinting & Identity (deviceAuth.ts)

When a mobile device loads the application for the first time, `frontend/lib/deviceAuth.ts` executes client-side identification:

1. Cryptographic Device ID Generation

```
let deviceId = localStorage.getItem("stocksense_device_id") || getCookie("stocksense_device_id");
if (!deviceId) {
  deviceId = `dev_${crypto.randomUUID()}`;
  localStorage.setItem("stocksense_device_id", deviceId);
  setCookie("stocksense_device_id", deviceId, 365);
}
```

- Uses the browser's hardware-backed **Web Crypto API** (`crypto.randomUUID()`), producing a collision-free UUIDv4 identifier.
- Uses **dual persistence**: both `localStorage` and a 1-year `SameSite=Lax` Cookie so the identifier survives browser tab closures, app refreshes, and cookie clears.

2. User-Agent Heuristic Labeling

To prevent the administrator from seeing incomprehensible raw UUID strings in the Admin Console, `deviceAuth.ts` analyzes `navigator.userAgent`:

```
export function detectDeviceFriendlyName(): string {
  const ua = navigator.userAgent;
  let os = /iPhone/i.test(ua) ? "iPhone" : /Android/i.test(ua) ? "Android" : ...;
  let browser = /Chrome\//i.test(ua) ? "Chrome" : /Safari\//i.test(ua) ? "Safari" : ...;
  return `${browser} on ${os}`; // e.g. "Safari on iPhone"
}
```

3. Request Header Injection

Once authorized, all outgoing fetch requests automatically attach three security headers:

```
headers["X-Device-ID"] = deviceId;
headers["X-Device-Token"] = secretToken;
headers["X-Client-Type"] = "remote-lan";
```

5. FastAPI Zero-Trust Perimeter Middleware (lan_middleware.py)

Every HTTP request that reaches FastAPI passes through `LANDeviceAccessMiddleware` (`backend/app/core/lan_middleware.py`).

1. Client IP Extraction

Because Next.js reverse-proxies requests to FastAPI, the immediate socket peer is `127.0.0.1`. The middleware extracts the true originating client IP by analyzing HTTP proxy headers:

```
def extract_client_ip(request: Request) -> str:
    forwarded = request.headers.get("x-forwarded-for")
    if forwarded:
        return forwarded.split(",")[0].strip() # Originating remote client IP
    real_ip = request.headers.get("x-real-ip")
    if real_ip:
        return real_ip.strip()
    return request.client.host
```

2. Whitelist of Exempt Paths

Discovery, health, and onboarding endpoints must be accessible without prior authorization:

```
EXEMPT_PATH_PREFIXES = (
    "/health",
    "/api/v1/health",
    "/api/v1/lan/status",
    "/api/v1/lan/devices/register",
    "/api/v1/lan/devices/check",
    "/docs",
    "/openapi.json",
)
```

3. Host Laptop Loopback Exemption (Anti-Lockout Protection)

To ensure the person sitting at the host laptop is never locked out of their own server:

```
if settings.lan_auto_approve_local and is_loopback(client_ip) and not is_remote_client:
    return await call_next(request)
```

- If the request originates from `127.0.0.1`, `::1`, or `localhost` **and** does not carry a `remote-lan` client header, it is immediately permitted.
- If an external phone passes through Next.js proxy, it carries `X-Client-Type: remote-lan` and `X-Forwarded-For: 192.168.1.55`. It is treated as an external client and **must** be authorized.

4. Device Verification & Token Cryptography

For all non-exempt endpoints:

- The middleware reads `X-Device-ID` and `X-Device-Token`.
- It queries `lan_devices` table in SQLite:
- If not found → Returns HTTP 403 `{"error": "DEVICE_NOT_REGISTERED"}`
- If status == `PENDING` → Returns HTTP 403 `{"error": "DEVICE_PENDING_APPROVAL"}`
- If status == `REJECTED` → Returns HTTP 403 `{"error": "DEVICE_REJECTED"}`
- If status == `REVOKED` → Returns HTTP 403 `{"error": "DEVICE_REVOKED"}`
- If status == `APPROVED`:
- It hashes the submitted token: `hash_token(raw_token)` using SHA-256.
- It verifies that `hash_token == device.token_hash`.
- If valid, access is granted (`call_next(request)`).

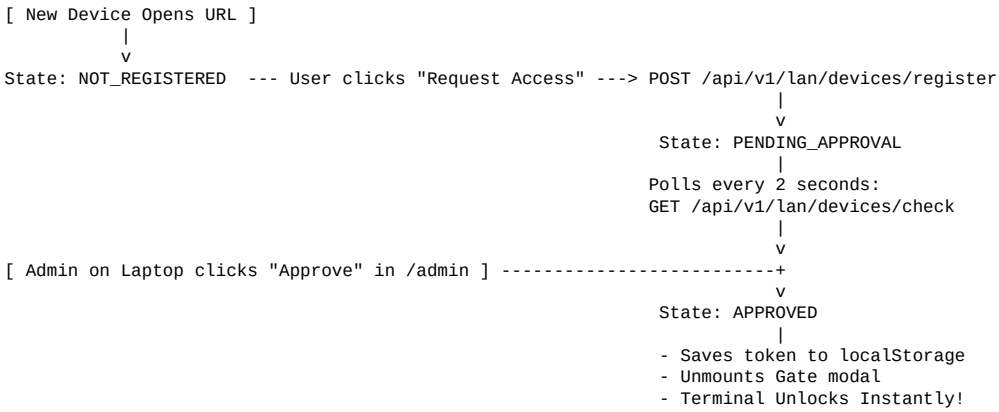
5. Persistent Audit Logging

Every attempt (allowed or blocked) is logged to `lan_access_logs`:

- Timestamp (UTC)
- Client IP
- Device ID
- HTTP Method & Path
- Status (`ALLOWED`, `BLOCKED`, `PENDING`)
- User-Agent

6. The Approval Workflow & Real-Time Polling (DeviceGate.tsx)

The UI workflow in `frontend/components/DeviceGate.tsx` operates as a state machine:



- **Registration**: The phone sends `device_id`, `device_name`, `client_ip`, and `user_agent` to `/api/v1/lan/devices/register`.
- **Database Record**: FastAPI creates a row in `lan_devices` with `status = "PENDING"`.
- **Polling**: The phone starts a `setInterval` timer polling `/api/v1/lan/devices/check?device_id=dev_...` every 2,000ms.

- **Admin Approval**: In `http://localhost:3000/admin`, the admin clicks **[Approve]**:
- Backend generates a cryptographically secure 256-bit token: `secrets.token_urlsafe(32)`.
- Backend hashes the token with SHA-256 and stores `token_hash` in `lan_devices`.
- Backend sets `status = "APPROVED"`.
- **Instant Unlock**: On the phone's next 2-second poll, the backend returns:

```
{
  "status": "APPROVED",
  "is_approved": true,
  "token": "sec_xK9mP2vL8..."
}
```

The phone saves the token to storage and immediately unmounts the gate screen. The full trading terminal is live!

7. Security Highlights & Threat Mitigations

Threat	How It Is Mitigated
Wi-Fi Eavesdroppers / Neighbors	Even if a neighbor knows your laptop's Wi-Fi IP, they cannot see any financial data. Their browser is blocked by the Device Gate until you manually click "Approve" in your private Admin Console.
Public Wi-Fi Exposure (Cafes / Airports)	The firewall rule specifies `Profile = "Private"`. On public networks, Windows automatically disables the rule and drops all inbound packets.
Database Compromise	Secret device tokens are never stored in plaintext. Only SHA-256 hashes are saved in `lan_devices.token_hash`.
Stolen Device Token / Lost Phone	The admin can click **[Revoke]** on `http://localhost:3000/admin`. The database status changes to `REVOKED` immediately, and the device is disconnected on its very next API request.
Host Lockout Risk	Loopback connections (`localhost`, `127.0.0.1`) originating directly on the server are auto-approved. You will never be locked out of your own computer.

8. Summary of Relevant Source Files

Firewall Config: `setup_windows_firewall.ps1 & setup_firewall.bat`

1-Click LAN Launcher: `start_stocksense_lan.bat`

FastAPI Middleware: `backend/app/core/lan_middleware.py`

Backend LAN Services: `backend/app/core/lan_service.py`

REST Endpoints: `backend/app/api/v1/endpoints/lan.py`

Database Models: `backend/app/models/lan.py`

Frontend Device Auth: `frontend/lib/deviceAuth.ts`

Frontend Access Gate Component: `frontend/components/DeviceGate.tsx`

Reverse Proxy Gateway: `frontend/next.config.js`

End of document.